// BLUEPRINT IMPLEMENTATION GUIDE — PART 2

PROJECT ANOMALY — Remaining Systems

Covers the 32 assets not in Part 1: enemy child BPs, projectiles, weapon overrides, skill system, arena manager, game instance, fodder AI, remaining HUD widgets, camera, and all missing structs/DataTables.

CONTINUES PART 1 | 32 ASSETS | FLOWCHART NODES | UE 5.4

// 01 REMAINING STRUCTS & DATATABLES

> Three structs and three DataTables weren't in Part 1. Create all of these before building the systems that depend on them — enemies, skills, arena waves.

CREATE FS_EnemyRow — struct for DT_Enemies

MEMBER	TYPE	PURPOSE
EnemyClass	Subclass of BP_EnemyBase	Which BP to spawn — set per row
MaxHealth	Float	Starting health pool
MaxArmour	Float	0 = no armour (Fodder, FlyingEye)
WeakPointCount	Int	How many WP actors to spawn
RagePerWeakPointHit	Float	Rage fed to player on each WP hit
bCanBeStunned	Bool	Whether AC_StunComponent is active
StunThreshold	Float	Sustained fire seconds needed to stun
bFiresProjectiles	Bool	Gates parry eligibility
SightRadius	Float	AIPerception detection range
BehaviourTree	Soft Object Ref to BehaviorTree	Assigned in AIC_ OnPossess

Create `DT_Enemies` in `Data/` using this struct. Add rows: `Brute`, `FlyingEye`, `Spawner`, `FodderMelee`, `FodderRanged`.

CREATE FS_SkillRow — struct for DT_Skills

MEMBER	TYPE	PURPOSE
SkillName	Text	Localisation-safe display name
SkillClass	Subclass of BP_SkillBase	Spawned by AC_SkillSlotComponent on activation
Description	Text	Shown in WBP_SkillSelectMenu
Icon	Soft Texture2D ref	Shown in HUD skill slot and menu
Duration	Float	Effect lifetime (0 = instant)
bRequiresEnragedState	Bool	Always true for current skills — flag for future non-rage skills

Create `DT_Skills` using this struct. Add one row per skill you build. The `WBP_SkillSelectMenu` reads all rows from this table automatically to populate the menu.

CREATE FS_WaveRow — struct for DT_ArenaWaves

MEMBER	TYPE	PURPOSE
WaveNumber	Int	Identifies the wave (for ordering)
EnemyRowName	Name	Row name in DT_Enemies to spawn
SpawnCount	Int	How many of this enemy to spawn
SpawnDelay	Float	Seconds before this batch spawns
bIsBossWave	Bool	Triggers special arena events

Create `DT_ArenaWaves` using this struct. Each row is one spawn event. A full wave can be multiple rows (e.g. row 1: 2 Brutes at 0s, row 2: 4 FodderMelee at 5s).

FLOW SUMMARY — REMAINING STRUCTS

1. **FS_EnemyRow** is read by BP_EnemyBase BeginPlay to initialise all stats — health, armour, stun threshold, WP count. One row per enemy type means tweaking the Brute is one cell edit, not a Blueprint change.
2. **FS_SkillRow** is the registry that makes the skill system open-ended. WBP_SkillSelectMenu iterates all rows in DT_Skills to build its list — adding a new skill requires only a new row and a new child BP.
3. **FS_WaveRow** drives BP_ArenaManager entirely. Each row is a timed spawn event. A designer can author entire arenas without touching any Blueprint graph.

🔧 DEBUG & TEST — STRUCTS & DATATABLES

HOW TO TEST

- **Verify DT_Enemies rows:** In BP_EnemyBase BeginPlay, print the RowName being queried. Each enemy subclass should print its own row name. If you see a blank or wrong name, the child BP's literal string is mistyped.
- **Test DT_Skills population:** In WBP_SkillSelectMenu's Event Construct, print the count of rows returned from DT_Skills. It should match how many skill rows you've added. Adding a row should increase the count without any code change.
- **Test DT_ArenaWaves ordering:** Add a `Print String` in SpawnBatch that prints the WaveNumber field. Rows should print in ascending WaveNumber order during an arena encounter.

EXPECTED RESULTS

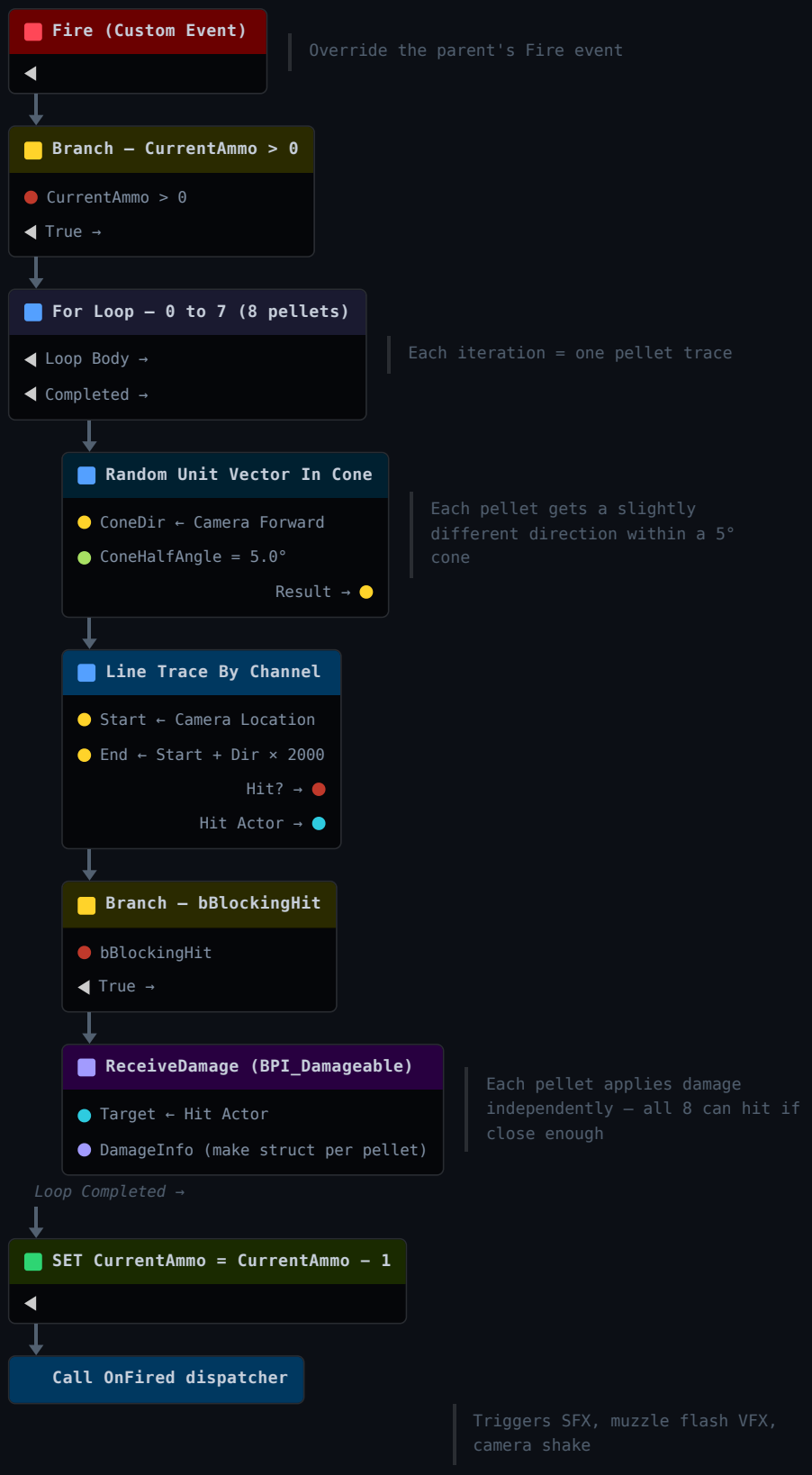
- ✓ PASS — Each enemy prints its correct row name on spawn
- ✓ PASS — DT_Skills row count matches entries in the table
- ✓ PASS — Wave batches print in correct order with correct delays
- ✗ FAIL — Row count is 0: DT_Skills DataTable asset reference is null. Set it as a variable default in WBP_SkillSelectMenu and assign the asset in defaults.

BP_WEAPON_SHOTGUN & BP_WEAPON_ASSAULTRIFLE

Both are child BPs of BP_WeaponBase. They only override `Fire()` — everything else (ammo, reload, multipliers) is inherited. Create each in `Weapons/` → Blueprint Class → parent = BP_WeaponBase.

GRAPH BP_Weapon_Shotgun — Fire() override: pellet spread

FIRE() — SPAWNS 8 SIMULTANEOUS HITSCAN TRACES IN A CONE



📌 **Penetration (up to 3 fodder):** After the trace hits an actor, call ReceiveDamage on it, then continue the same trace from the hit location forward. Use a counter variable — if PenetrationCount < 3 and the hit actor is a

```
    fodder type (check tag or cast), re-run the trace from hit location to keep going through the group.
```

GRAPH

BP_Weapon_AssaultRifle — Fire() override: wind-up rate + movement penalty

Add extra variables: `CurrentFireInterval` (Float, default 0.5 — slow), `MinFireInterval` (Float, default 0.08 — fast), `WindUpRate` (Float, default 0.05), `bIsFiring` (Bool), `FireTimerHandle` (FTimerHandle).

FIRE() — STARTS A LOOPING TIMER THAT ACCELERATES OVER TIME

Branch — NOT bIsFiring
- NOT bIsFiring
- ◀ True → start firing

> Prevents stacking multiple timers

↓

SET bIsFiring = true
- ◀

↓

Get Owner → Cast → BP_PlayerCharacter
- ◀ Success →

> Need the character movement component for the speed penalty

↓

Get Character Movement → SET MaxWalkSpeed = 200
- ◀

> Reduce movement speed while the rifle is firing (restored in StopFire)

↓

Set Timer by Function Name "DoFireTick"
- Time ← CurrentFireInterval
- Looping: false
- → SET FireTimerHandle

> Not looping — DoFireTick re-schedules itself with a shorter interval each time

DOFIRETICK CUSTOM EVENT — CALLED BY THE TIMER EACH SHOT

Custom Event: DoFireTick
- ◀

↓

Branch — bIsFiring AND CurrentAmmo > 0
- ◀ True →
- ◀ False → StopFire

↓

Line Trace By Channel (single shot)
- Camera forward, 5000 units

↓

FInterp To
- Current ← CurrentFireInterval
- Target ← MinFireInterval
- DeltaTime ← WindUpRate
- → SET CurrentFireInterval

> Each shot decreases the interval — gun "spins up" to full rate

↓

Set Timer → "DoFireTick", Time=CurrentFireInterval

> Re-schedule with the new (shorter) interval — self-accelerating loop

STOPFIRE CUSTOM EVENT — CALLED ON INPUT RELEASE

SET bIsFiring = false
- ◀

↓

SET CurrentFireInterval = 0.5 (reset wind-up)
- ◀

↓

Clear Timer by Handle (FireTimerHandle)

Cancel any pending fire tick

↓

Character Movement → SET MaxWalkSpeed = 600 (restore)

Restore normal movement speed when trigger released

📌 **Wiring StopFire in BP_PlayerCharacter:** In the Enhanced Input graph, the IA_Fire action has two outputs — **Started** (calls WeaponComponent.Fire) and **Completed** (calls the active weapon's StopFire custom event via a cast). Only the AR has a StopFire — Pistol and Shotgun can ignore it.

FLOW SUMMARY — SHOTGUN & ASSAULT RIFLE

1. **Shotgun spread:** A For Loop runs 8 iterations. Each one generates a random direction within a 5° cone using Random Unit Vector In Cone, fires a line trace, and independently deals damage. All 8 pellets run in the same frame.
2. **AR wind-up:** Instead of a simple looping timer, DoFireTick re-schedules itself each shot with a shorter interval using FInterp To. This creates a smooth acceleration curve from slow to fast. When the trigger releases, StopFire cancels the pending timer and resets the interval back to slow.
3. **Movement penalty:** On fire start, MaxWalkSpeed on CharacterMovement is reduced to 200. On StopFire, it restores to 600. This is the GDD mechanic: sustained fire slows the player.
4. **Stun contribution:** The AR also calls AC_StunComponent.AddStunDamage on whatever it hits each tick — this is how sustained fire builds the stun accumulator on enemies like the Brute and FlyingEye.

🔧 DEBUG & TEST — SHOTGUN & ASSAULT RIFLE

HOW TO TEST

- **Visualise shotgun spread:** Add `Draw Debug Line` inside the shotgun's For Loop after each Line Trace — same start/end points, red colour, 1s duration. Fire at a wall from 5m away. You should see 8 spread lines forming a cone pattern.
- **Test AR wind-up:** Print `CurrentFireInterval` every shot tick. It should start at 0.5 and decrease toward MinFireInterval with each shot. Release and refire — should start slow again.
- **Test AR movement penalty:** Print `GetCharacterMovement().MaxWalkSpeed` before firing (600) and after firing starts (200). Release trigger — should return to 600.
- **Test shotgun penetration:** Line up 3 FodderMelee. Fire the shotgun. All 3 should take damage from a single pellet that passes through them.

EXPECTED RESULTS

- ✓ PASS — 8 debug lines visible in a cone on shotgun fire
- ✓ PASS — AR interval: 0.5 → decreasing → MinFireInterval over sustained fire
- ✓ PASS — MaxWalkSpeed: 600 → 200 on fire start → 600 on release
- ✓ PASS — All 3 lined-up fodder take damage from one penetrating pellet
- ✗ FAIL — AR never stops firing after input release: StopFire Custom Event is not being called on input Completed. Check the IA_Fire Completed binding in BP_PlayerController.

// 03

BP_PROJECTILE_BASE + BP_PROJECTILE_ENEMY

BP_Projectile_Base is the abstract parent for all projectiles. It implements BPI_Parryable. BP_Projectile_Enemy extends it with the green highlight window that makes it parryable by the player.

SETUP BP_Projectile_Base — create and configure

In `Combat/` → Blueprint Class → parent = **Actor**. Name `BP_Projectile_Base`. Check **Abstract** in Class Settings. Add components: **Sphere Collision** (radius 15), **Static Mesh** (sphere), **Projectile Movement Component** (InitialSpeed=2000, MaxSpeed=2000, bShouldBounce=false). Add variables:

VARIABLE	TYPE	DEFAULT
Damage	Float	20.0
bCanBeParried	Bool	false — set true in BP_Projectile_Enemy
Instigator_Enemy	BP_EnemyBase ref	Set on spawn by enemy AI task

Implement `BPI_Parryable` on this BP. Wire `CanBeParried` to return `bCanBeParried`. Wire `OnParried` to **Destroy Actor**. Wire `OnPerfectParried` to Destroy Actor AND call `AC_StunComponent.AddStunDamage` on `Instigator_Enemy` with a large value to instantly stun.

GRAPH BP_Projectile_Base — On Hit: deal damage and destroy

SPHERE COLLISION → ON COMPONENT HIT

GRAPH

BP_Projectile_Enemy — parry highlight window

Create `BP_Projectile_Enemy` in `Combat/` → parent = **BP_Projectile_Base**. Add variable `ParryWindowDuration` (Float, default 0.6). Add a **Material Instance Dynamic** variable for the highlight material. Wire BeginPlay:

BEGINPLAY — OPEN PARRY HIGHLIGHT WINDOW AFTER SHORT TRAVEL DELAY

Event BeginPlay

Projectile just spawned — give it a short travel time before becoming parryable

Set Timer → "OpenParryWindow", Time=0.3s

0.3s travel time before the projectile becomes green/parryable

OPENPARRYWINDOW CUSTOM EVENT

SET bCanBeParried = true

BPI_Parryable.CanBeParried now returns true

Set Scalar Parameter Value on Mesh Material

- ParameterName: "GlowIntensity"
- Value: 1.0

Turn on the green highlight glow — player can now see it's parryable

Set Timer → "CloseParryWindow", Time=ParryWindowDuration

Window closes after 0.6s — projectile becomes un-parryable again

CLOSEPARRYWINDOW CUSTOM EVENT

SET bCanBeParried = false

Set Scalar Parameter — GlowIntensity = 0.0

Turn off the glow — grey/normal projectile again

FLOW SUMMARY — PROJECTILE SYSTEM

1. **BP_Projectile_Base** handles the universal logic: Projectile Movement drives flight, On Component Hit applies BPI_Damageable damage to whatever it contacts, then self-destructs. Abstract so it's never placed directly.
2. **Parry window lifecycle:** BeginPlay → 0.3s delay → bCanBeParried=true + green glow → 0.6s window → bCanBeParried=false + glow off. The window exists independently of BP_ParryWindow — the projectile manages its own parryability state.
3. **BPI_Parryable implementation:** CanBeParried just returns bCanBeParried. OnParried destroys the projectile. OnPerfectParried destroys it AND calls a large AddStunDamage on the enemy that fired it — triggering an instant stun.
4. **Instigator_Enemy reference:** Set by BTT_FireProjectile when spawning the projectile. This is how OnPerfectParried knows which enemy to stun without a global search.

🔧 DEBUG & TEST — PROJECTILE SYSTEM

HOW TO TEST

- **Visualise parry window:** Add `Print String "PARRY OPEN"` in OpenParryWindow and `Print String "PARRY CLOSED"` in CloseParryWindow. Watch the timing — "OPEN" should appear 0.3s after the projectile spawns, "CLOSED" 0.6s later.
- **Test green glow toggle:** The projectile mesh material should visibly glow green during the parry window and return to normal after. If no glow: check the GlowIntensity parameter name matches exactly in both the material and the Set Scalar node.
- **Test perfect parry stun:** Land a perfect parry on a FlyingEye projectile. The FlyingEye should immediately stagger/stun. Print `"PERFECT PARRY STUN"` in AC_StunComponent.OnStunBegin to confirm the chain.
- **Test self-collision prevention:** Enemy fires a projectile. It should not immediately destroy itself on spawn. The 80-unit offset in BTT_FireProjectile prevents this — verify by checking that projectiles travel at least 1 full second before hitting anything.

EXPECTED RESULTS

- ✓ PASS — "PARRY OPEN" prints 0.3s after spawn, "CLOSED" 0.6s after open
- ✓ PASS — Projectile glows green during window, normal outside
- ✓ PASS — Perfect parry: "PERFECT PARRY STUN" prints, enemy staggers
- ✗ FAIL — Projectile destroys itself instantly on spawn: spawn offset is zero or negative. Increase the offset multiplier in BTT_FireProjectile.
- ✗ FAIL — bCanBeParried always true: OpenParryWindow is being called at spawn instead of after the 0.3s delay. Check the timer in BeginPlay.

BP_SKILLBASE + BUILDING CHILD SKILLS

> BP_SkillBase is the abstract parent. Child skills only override Execute(). This section covers the parent setup and shows how to build one concrete skill as an example — the same pattern applies to every future skill.

SETUP BP_SkillBase — abstract parent

In `Skills/Base/` → Blueprint Class → parent = **Actor**. Name `BP_SkillBase`. Check **Abstract**. Implement `BPI_SkillEquippable`. Add variables:

VARIABLE	TYPE	NOTES
SkillDataRowName	Name	Row key in DT_Skills — set in child BP defaults
OwnerPawn	Pawn ref	Set by AC_SkillSlotComponent when spawning
Duration	Float	Read from DT_Skills row at BeginPlay

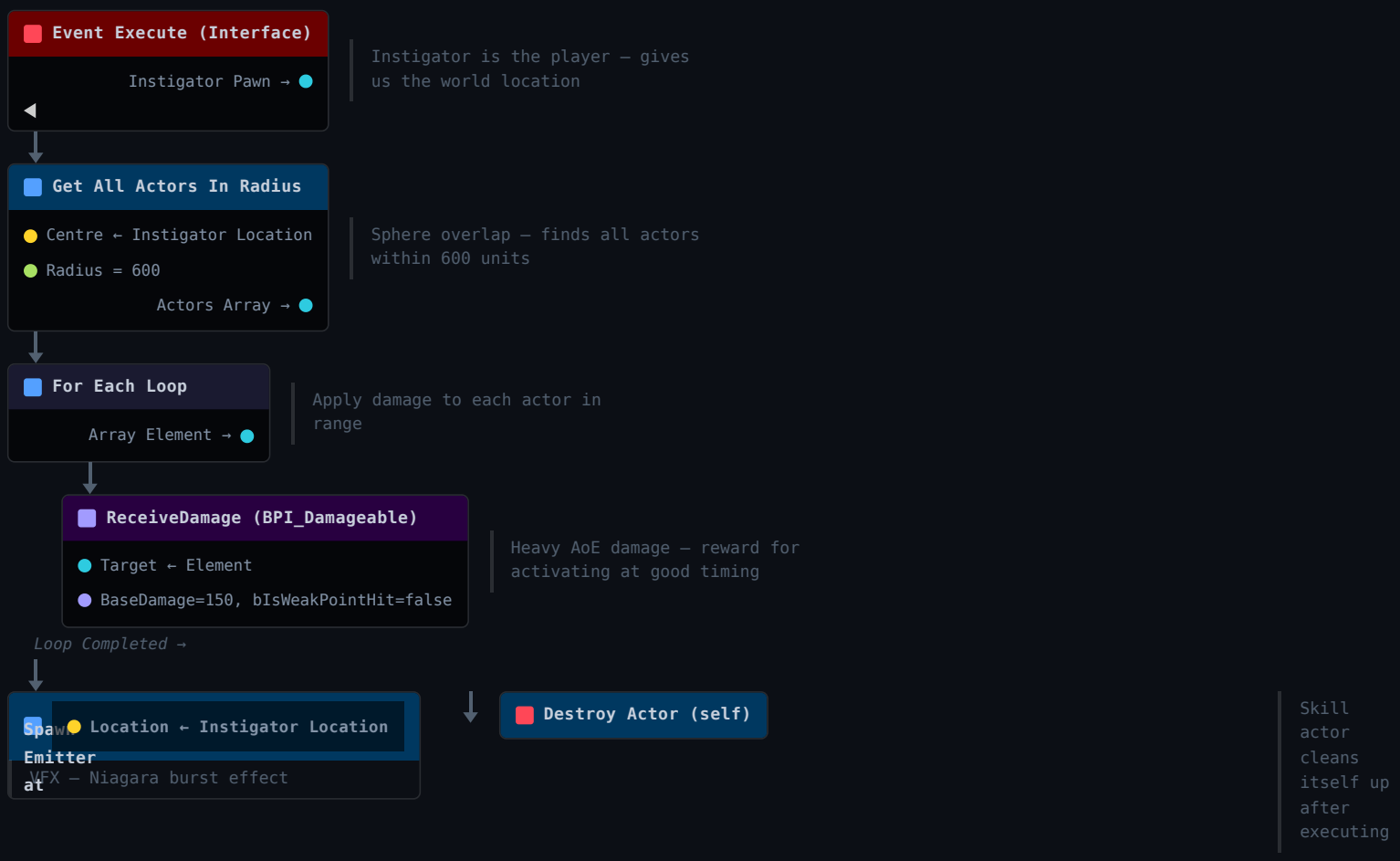
Wire the `BPI_SkillEquippable` → `Execute` interface implementation as an empty event — child BPs override it with their specific logic. Wire `GetSkillData` to return the DA_SkillData for the equipped row.

EXAMPLE

BP_Skill_BlastWave — example child skill

In `Skills/` → Blueprint Class → parent = **BP_SkillBase**. Name `BP_Skill_BlastWave`. This skill fires a radial burst from the player when activated. Wire the Execute() interface override:

EXECUTE (BPI_SKILLEQUIPPABLE OVERRIDE) — FIRES WHEN AC_SKILLSLOTCOMPONENT CALLS IT



💡 **Adding any new skill in the future:** 1) Create a child BP of BP_SkillBase. 2) Override the Execute() interface event with your logic. 3) Add a row to DT_Skills pointing to this new class. 4) It automatically appears in WBP_SkillSelectMenu at the next save point. Zero changes to existing code.

FLOW SUMMARY — BP_SKILLBASE + CHILD SKILLS

1	**Abstract parent:**	BP_SkillBase holds variables and implements BPI_SkillEquippable with an empty Execute(). It never does anything on its own — it's a contract and shared data carrier.
2	**Child overrides Execute():**	Each skill BP overrides just the Execute interface event and puts its unique logic there. Everything else is inherited.
3	**Spawn and destroy pattern:**	AC_SkillSlotComponent spawns the skill actor → calls Execute via interface → the skill does its thing → self-destructs. The slot component never knows what the skill did.
4	**DT_Skills as registry:**	The SkillClass column in DT_Skills holds the class reference. AC_SkillSlotComponent reads this at activation time to know what to spawn. Adding a skill = adding a row.

🔧 DEBUG & TEST — BP_SKILLBASE + CHILD SKILLS

HOW TO TEST

- **Test skill spawn:** Add `Print String "SKILL SPAWNED: " + SkillDataRowName` in BP_SkillBase's Execute interface implementation (or override). Activate during enrage — confirm the correct skill name prints.
- **Test skill self-destructs:** After Execute runs, the skill actor should be gone. Add `Print String "SKILL ALIVE"` to BP_SkillBase Event Tick. It should print briefly then stop. If it keeps printing, Destroy Actor isn't being called at the end of Execute.
- **Test new skill registration:** Add a new row to DT_Skills pointing to a new child skill BP. Without changing any other code, interact with a save point — the new skill should appear in the menu automatically.
- **Test BlastWave damage radius:** Place enemies at known distances. Activate BlastWave. Enemies at r<600 take damage, enemies at r>600 do not.

EXPECTED RESULTS

- ✓ PASS — Correct skill name prints on activation
- ✓ PASS — "SKILL ALIVE" tick print stops within the skill's duration
- ✓ PASS — New DT row = new skill appears in menu without code changes
- ✓ PASS — BlastWave: enemies within 600 units take damage, others unaffected
- ✗ FAIL — Skill not in menu: SkillClass column in DT_Skills is null. Open the row and assign the child BP class reference.

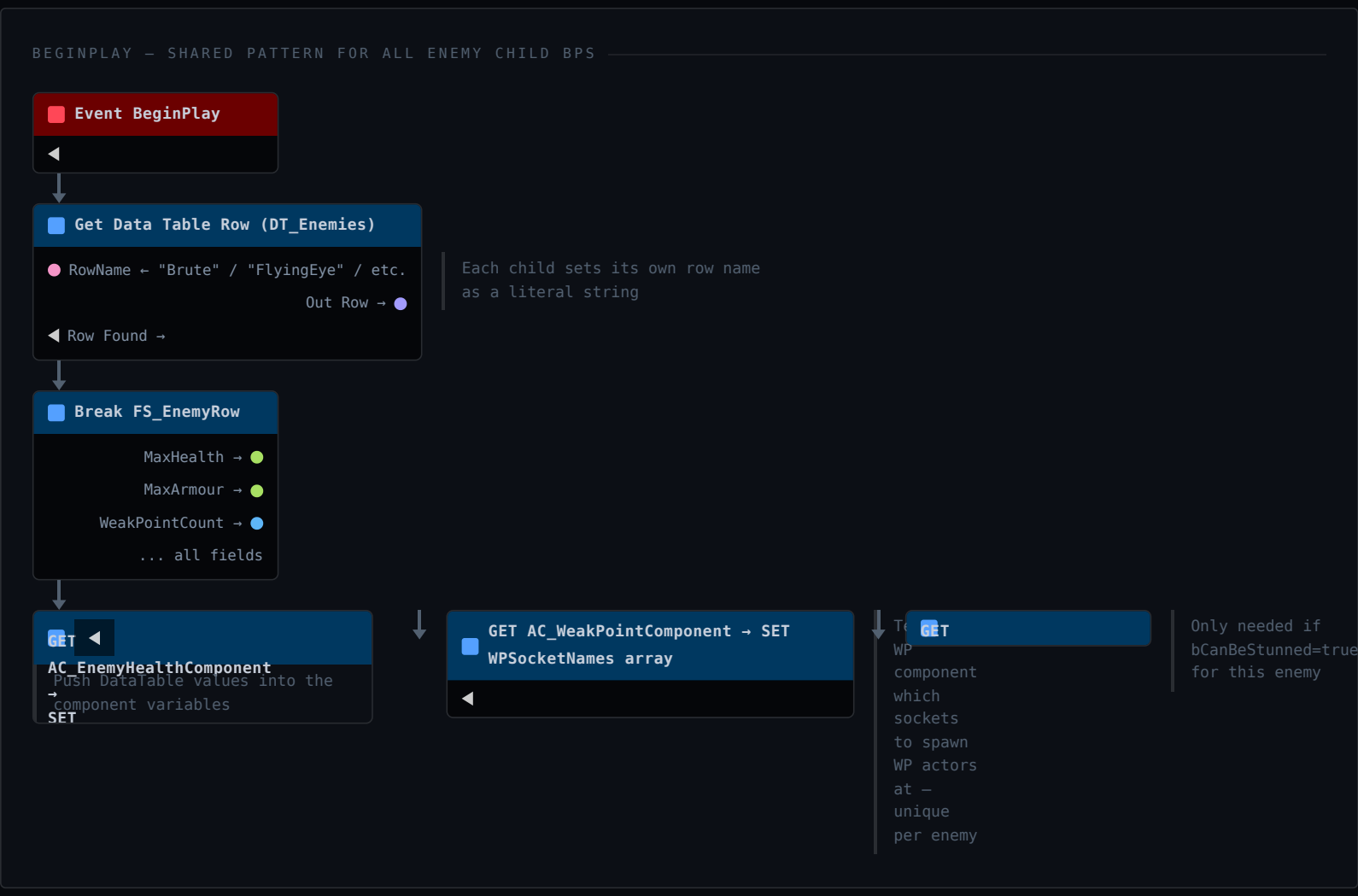
// 05

ENEMY CHILD BPS — FLYINGEYE, SPAWNER, FODDER

These are all children of BP_EnemyBase. They inherit every shared component. Each child BP only adds unique variables, overrides BeginPlay to load its DT_Enemies row, and sets up its unique component configuration.

SETUP How to create any enemy child BP — common pattern

In the enemy's subfolder → Blueprint Class → parent = **BP_EnemyBase**. Open the child BP. In **Class Defaults**, set `AI Controller Class` to the matching AIC_. Then wire BeginPlay to load stats from DT_Enemies:



UNIQUE BP_Enemy_FlyingEye — eye weak point immobilises

In BP_Enemy_FlyingEye's BeginPlay, after the shared setup, also bind to AC_WeakPointComponent's `OnWeakPointHit` delegate. When WeakPointIndex = 0 (the eye), write to the Blackboard:

UNIQUE

BP_Enemy_Spawner — connect AC_SpawnDirectorComponent

BP_Enemy_Spawner adds `AC_SpawnDirectorComponent` in its Components panel. In BeginPlay, after the shared setup, initialise the spawn director:

BP_ENEMY_SPAWNER — ADDITIONAL BEGINPLAY WIRING

GET AC_SpawnDirectorComponent
Ref →

↓

SET MaxSpawnBudget = 6, CurrentBudget = 6
◀

> Can spawn 6 fodder total before budget runs out

↓

Bind Event to AC_EnemyHealthComponent.OnDeath
● Event ← Custom Event: OnSelfDeath

> When the Spawner itself dies, stop spawning

AC_SPAWNDIRECTORCOMPONENT — TRYSPAWNFODDER FUNCTION

Branch — CurrentBudget > 0
◀ True →
◀ False → write bCanSpawn=false to BB

↓

Spawn Actor (random fodder type)
● Location ← random spawn point from array
Spawned Fodder → ●

> Randomly pick Melee or Ranged fodder class

↓

Bind to Fodder.OnDeath → OnFodderDied
● Target ← Spawned Fodder's HealthComp

> When this fodder dies, refill 1 budget slot

↓

SET CurrentBudget = CurrentBudget - 1
◀

↓

Set BB Bool "bCanSpawn" ← CurrentBudget > 0

> Update Blackboard so BTD_bCanSpawn decorator reflects current budget state

SETUP BP_Enemy_FodderMelee + BP_Enemy_FodderRanged

Both are children of BP_EnemyBase. They have **no weak points** (WeakPointCount=0 in their DT row) so AC_WeakPointComponent spawns nothing. Key difference: FodderRanged adds a spawn for BP_Projectile_Enemy on its attack task instead of using BTT_AttackMelee.

Set `AI Controller Class = AIC_Fodder` on both. AIC_Fodder is shared — it runs BT_Fodder_Melee or BT_Fodder_Ranged depending on which pawn it possesses (detected via Cast in OnPossess).

FLOW SUMMARY — ENEMY CHILD BPS

1. **Common BeginPlay pattern:** Every enemy child BP reads its row from DT_Enemies and pushes values into its inherited components. The child BP's only unique job is knowing its own row name and which sockets to assign for weak points.
2. **FlyingEye eye hit:** Binding to OnWeakPointHit in the child BP means only the FlyingEye reacts to its eye being hit — the base class and other enemies are unaffected.
3. **Spawner budget:** AC_SpawnDirectorComponent tracks a finite pool. Each spawn decrements it; each fodder death increments it. When budget hits 0, bCanSpawn=false blocks the BT spawn task immediately via the BTD_ decorator.
4. **Fodder simplicity:** Fodder has zero weak points and no stun — their BeginPlay is just the DataTable load. AIC_Fodder chooses the correct Behavior Tree based on which subclass it possesses.

🔧 DEBUG & TEST — ENEMY CHILD BPS

HOW TO TEST

- **Verify stats loaded from DT:** During PIE, select a spawned Brute → Details panel → find AC_EnemyHealthComponent → inspect MaxHealth and MaxArmour. They should match the DT_Enemies "Brute" row values.
- **Test FlyingEye immobilise:** Shoot the eye WP (index 0). The FlyingEye should immediately stop moving. Print "EYE DESTROYED" in the OnWPHit_Eye event to confirm it fires. Check bImmobilised=true in the Blackboard Debugger.
- **Test Spawner budget:** Set MaxSpawnBudget=2 temporarily. Kill two fodder — SpawnBudget should reach 0, bCanSpawn=false in BB Debugger, and no more fodder spawns.
- **Test AIC_Fodder tree selection:** Spawn a FodderMelee and a FodderRanged near the player. Melee should charge; Ranged should keep distance and fire. If both behave the same, the Cast-based BT selection in AIC_Fodder is not working.

EXPECTED RESULTS

- ✓ PASS — Brute MaxHealth/MaxArmour match DT_Enemies row values exactly
- ✓ PASS — Eye hit: FlyingEye freezes, bImmobilised=true in BB Debugger
- ✓ PASS — Budget=0: no new fodder spawns until existing ones die
- ✓ PASS — Melee charges, Ranged keeps distance and fires
- ✗ FAIL — Stats are zero defaults: Get Data Table Row isn't finding the row. Print the row name being queried — check for spelling differences between the child BP's literal and the DT row name.

BT_FODDER TREES + REMAINING SHARED BT NODES

BT_Fodder_Melee — charge and swipe

NODE	TYPE / CONFIG	NOTES
Root → Sequence	S: BTS_UpdatePlayerLocation	Always tracking
├ BTT_FindPlayerLocation	Task — writes TargetLocation	
├ Move To	Key=TargetLocation, AcceptRadius=80	Get very close — melee range
└ BTT_AttackMelee	D: BTD_PlayerInRange (range=120)	Only swing if player is close enough

BT_Fodder_Ranged — maintain distance and fire

NODE	TYPE / CONFIG	NOTES
Root → Selector	S: BTS_UpdatePlayerLocation, S: BTS_CheckLineOfSight	Both services always ticking
├ Sequence	D: BTD_PlayerInRange (InRange=true, max=150) — too close, back off	If player is too close, retreat first
│ └ Move To	Away from player — negate direction vector	Step backward to preferred range
├ Sequence	D: BTD_PlayerInRange (min=200, max=600) + D: BB bHasLOS=true	In range AND has LOS — fire
│ └ BTT_FireProjectile	Target=TargetActor	Fires BP_Projectile_Enemy
└ BTT_FindPlayerLocation	Fallback move	

BT NODE

BTT_MoveToEQSResult — custom task

This task is used by FlyingEye and Spawner after their EQS queries write a vector to the Blackboard. Create as BT Task in `Enemies/Shared_AI/`. Add an input variable `TargetKey` (BlackboardKeySelector type) so the same task can be reused with different BB keys.

BTT_MOVETOEQSRESULT — RECEIVE EXECUTE AI

Receive Execute AI

Owner Controller →

↓

Get Blackboard → Get Value as Vector

KeyName ← TargetKey variable

Vector Value →

Read the EQS result that was written to the BB

↓

Move To Location (via AIController)

Destination ← Vector Value

AcceptanceRadius = 50

Result (EPathFollowing) →

UE's built-in nav mesh pathfinding to the destination

↓

Branch — Result == Success

True →

False →

REACHED

Finish Execute (Success=true)

BLOCKED

Finish Execute (Success=false)

BT NODE

BTD_PlayerInRange — decorator

Create as BT Decorator in `Enemies/Shared_AI/`. Add variables: `MinRange` (Float, default 0), `MaxRange` (Float, default 300). Wire `Perform Condition Check AI`:

PERFORM CONDITION CHECK AI — RETURNS TRUE IF PLAYER IS WITHIN RANGE BAND

Get Controlled Pawn → Get Actor Location
- Enemy Location →

↓

Get Player Character → Get Actor Location
- Player Location →

↓

Vector Distance
- A ← Enemy Location
- B ← Player Location
- Distance →

↓

In Range (Float)
- Value ← Distance
- Min ← MinRange
- Max ← MaxRange
- → Return Node

> Returns true only if distance falls between Min and Max — usable as "too close" OR "attack range" by changing the values in each BT node

FLOW SUMMARY — FODDER BTS + SHARED NODES

1. **BT_Fodder_Melee:** Single-purpose tree. Find player → close gap → swing. BTD_PlayerInRange gates the swing to prevent attacking through walls or at range.
2. **BT_Fodder_Ranged:** Three-state tree: if too close → back off, if in range with LOS → fire, otherwise → pursue. The Selector naturally handles priority — backing off always takes precedence over firing.
3. **BTT_MoveToEQSResult:** A reusable task that takes a BlackboardKeySelector input, so the same task works for the FlyingEye's StrafeDestination and the Spawner's RetreatDestination without duplication.
4. **BTD_PlayerInRange:** One decorator, two uses. Set MinRange=0, MaxRange=120 for melee gate. Set MinRange=200, MaxRange=600 for ranged fire gate. The same node handles both cases with different configured values in the BT editor.

🔧 DEBUG & TEST — FODDER BTS + BTT_MOVETOEQSRESULT + BTD_PLAYERINRANGE

HOW TO TEST

- **Test melee fodder charge:** Open BT_Fodder_Melee Debugger. The Move To node should stay green until the fodder gets within 80 units, then BTT_AttackMelee activates. BTD_PlayerInRange decorator should show as passing (green outline) only when in range.
- **Test ranged fodder retreat:** Walk directly into a FodderRanged within 150 units. It should immediately back away. The retreat Sequence should show green in BT Debugger, other branches grey.
- **Test BTT_MoveToEQSResult failure path:** Block the navmesh path to the EQS destination (place a wall). The task should call Finish Execute with Success=false and the Selector tries its next child, rather than freezing.
- **Test BTD_PlayerInRange values:** Hover over the decorator in BT Debugger during PIE — the tooltip shows current distance and whether the condition is passing. Confirm Min/Max range values match what you set in the BT node.

EXPECTED RESULTS

- ✓ PASS — Melee: Move To green → attack activates on range entry
- ✓ PASS — Ranged: retreat branch green when player is within 150 units
- ✓ PASS — Blocked path: BT continues, does not freeze
- ✓ PASS — Decorator tooltip shows correct live distance
- ✗ FAIL — Ranged fodder doesn't retreat: ensure retreat Sequence is the FIRST child of the Selector (leftmost = highest priority). A later child can never override an earlier one that's passing.

BTT_FIREPROJECTILE + BTT_SPAWNFODDER

BT TASK BTT_FireProjectile

BTT_FIREPROJECTILE — RECEIVE EXECUTE AI

Receive Execute AI
Controlled Pawn →

↓

Get BB → Get Value as Object "TargetActor"
Target (Player) →

> Read the player reference from Blackboard — don't hardcode

↓

Direction = (Target Location − Pawn Location).Normalize
Direction →

> Calculate aim direction toward player

↓

Spawn Actor BP_Projectile_Enemy
Location ← Pawn Location + Direction × 80
Rotation ← Make Rot from Direction
Projectile Ref →

> Offset spawn so it doesn't immediately hit the enemy itself

↓

Cast → BP_Projectile_Enemy → SET Instigator_Enemy ← Pawn

> Store the firing enemy so OnPerfectParried can stun it

↓

Finish Execute (Success=true)

BT TASK BTT_SpawnFodder

BTT_SPAWNFODDER — DELEGATES TO AC_SPAWNDIRECTORCOMPONENT

Receive Execute AI
Controlled Pawn →

↓

Cast Pawn → BP_Enemy_Spawner
◀ Success →

↓

GET AC_SpawnDirectorComponent → TrySpawnFodder()
◀

> All spawn logic lives in the component — this task just calls it

↓

Finish Execute (Success=true)

FLOW SUMMARY — BTT_FIREPROJECTILE + BTT_SPAWNFODDER

1. **BTT_FireProjectile** reads the target from the Blackboard (not from a hardcoded player reference), calculates the aim direction, spawns the projectile slightly in front of the enemy so it doesn't self-collide, and sets the Instigator_Enemy reference on the projectile for use during OnPerfectParried.

2 **BTT_SpawnFodder** is intentionally thin — it just casts the pawn to BP_Enemy_Spawner and calls TrySpawnFodder() on the component. All budget tracking and spawn logic lives in AC_SpawnDirectorComponent. This keeps tasks simple and components testable.

3 **Always Finish Execute:** Both tasks call Finish Execute unconditionally. If the task couldn't complete (e.g. cast failed), it still calls Finish Execute with Success=false so the BT continues rather than freezing.

🔧 DEBUG & TEST — BTT_FIREPROJECTILE + BTT_SPAWNFODDER

HOW TO TEST

- **Test projectile aim:** Stand still and let an enemy fire. The projectile should travel directly toward your position at the moment of firing. Add `Draw Debug Line` from spawn location to target location (green) to visualise the aim vector.
- **Test Instigator_Enemy reference:** After a perfect parry, print the Instigator_Enemy reference in the OnPerfectParried handler. It should print the enemy actor's name, not None/null.
- **Test spawn budget decrement:** Watch the SpawnBudget BB key in the Blackboard Debugger while BTT_SpawnFodder fires. It should decrement by 1 each time fodder spawns.
- **Test BT freeze protection:** Temporarily remove the Finish Execute from BTT_SpawnFodder. The Spawner's BT should freeze on that task (verify in BT Debugger). Then restore it — the BT should resume immediately.

EXPECTED RESULTS

- ✓ PASS — Debug aim line points accurately from enemy to player
- ✓ PASS — Instigator_Enemy is valid (not null) after perfect parry
- ✓ PASS — SpawnBudget decrements correctly in BB Debugger
- ✗ FAIL — Projectile aims in wrong direction: the normalise node is missing. The direction vector must be normalised before multiplying by spawn offset distance.

BP_ARENAMANAGER + BP_ENEMYSPAWNER

> BP_ArenaManager orchestrates waves from DT_ArenaWaves. BP_EnemySpawner is a placed actor that marks valid spawn locations in the level. They work together — the manager picks spawner actors and tells them what to spawn.

SETUP BP_EnemySpawner — placed actor

In `Arena/` → Blueprint Class → parent = Actor. Name `BP_EnemySpawner`. Add a **Billboard Component** (visible in editor only) and a variable `SpawnerTag` (Name, e.g. "SpawnZone_A"). That's all — this is just a marker. Place multiple of these around your arena level.

GRAPH

BP_ArenaManager — wave system

Create in `Arena/` → Blueprint Class → parent = Actor. Place one in the level. Add variables: `CurrentWaveIndex` (Int), `ActiveEnemies` (Array of BP_EnemyBase), `bArenaActive` (Bool). Add Event Dispatcher `OnWaveComplete` and `OnArenaComplete`.

STARTWAVE FUNCTION — CALL THIS TO BEGIN AN ARENA ENCOUNTER

SET bArenaActive = true, CurrentWaveIndex = 0

◀

↓

Get All Rows from DT_ArenaWaves

Row Names Array →

Get all wave rows — we'll iterate through them in order

↓

For Each Loop — Row Names

Row Name →

↓

Get Data Table Row (DT_ArenaWaves)

RowName ← current row

FS_WaveRow →

↓

Set Timer ● Looping: false

→ "SpawnBatch",

Each wave row spawns after its own delay — creates staggered waves

SPAWNBATCH CUSTOM EVENT — FIRES PER WAVE ROW AFTER DELAY

For Loop — 0 to Row.SpawnCount

◀ Loop Body →

↓

Get All Actors Of Class BP_EnemySpawner → Pick Random

Random Spawner →

Pick a random placed spawner from the level

↓

Spawn Actor (Row.EnemyClass)

Location ← Spawner Location

Enemy Ref →

↓

Bind to Enemy HealthComp.OnDeath → OnEnemyDied

Track each spawned enemy's death to check if wave is complete

↓

Array ADD → ActiveEnemies

Track how many enemies are still alive

ONENEMYDIED CUSTOM EVENT — BOUND TO EACH ENEMY'S ONDEATH

Array REMOVE ◀

↓

Branch — ActiveEnemies.Num == 0

◀ True → all dead

↓

Call OnWaveComplete or OnArenaComplete

Open doors, unlock save point, trigger story

event
etc.

FLOW SUMMARY — BP_ARENAMANAGER

1. **StartWave reads DT_ArenaWaves:** Every row is a spawn batch with its own delay. The manager loops through all rows on start, setting a timer per row. When each timer fires, it spawns that batch at a random BP_EnemySpawner location.
2. **Death tracking:** Every spawned enemy gets an OnDeath binding. When an enemy dies, it's removed from ActiveEnemies array. When that array hits 0, the wave (or arena) is complete.
3. **Designer control:** Changing wave composition, enemy count, spawn timing — all done in DT_ArenaWaves without opening Blueprint. Level designers place BP_EnemySpawner actors to define the spawn zone, no Blueprint needed there either.

🔧 DEBUG & TEST — BP_ARENAMANAGER

HOW TO TEST

- **Test wave timing:** Add a `Print String "SPAWNING WAVE ROW: " + RowName` in SpawnBatch. Watch the prints during an arena encounter — each should appear at the correct SpawnDelay offset from when StartWave was called.
- **Test enemy count tracking:** Print `ActiveEnemies.Num()` after each spawn (should increase) and after each OnEnemyDied fires (should decrease). When it hits 0, OnWaveComplete should fire.
- **Test OnWaveComplete:** Add a `Print String "WAVE COMPLETE"` to OnWaveComplete. Kill all spawned enemies — the print should appear and doors/triggers should activate.
- **Test random spawner selection:** Place 3 BP_EnemySpawner actors. Run the arena multiple times. Enemies should appear at different spawner locations across runs (random selection).

EXPECTED RESULTS

- ✓ PASS — Wave rows print at correct delays after StartWave
- ✓ PASS — ActiveEnemies.Num() tracks correctly up and down
- ✓ PASS — "WAVE COMPLETE" prints when last enemy dies
- ✓ PASS — Enemies appear at varied spawner locations
- ✗ FAIL — OnWaveComplete never fires: check OnEnemyDied is bound to every enemy's HealthComp.OnDeath. If the bind happens after the enemy is already dead, it won't fire.
- ✗ FAIL — All enemies spawn at same location: Get All Actors Of Class is returning only one result. Ensure multiple BP_EnemySpawner actors are placed in the level.

BP_GI_ANOMALY (GAME INSTANCE) + BP_GS_ANOMALY (GAME STATE)

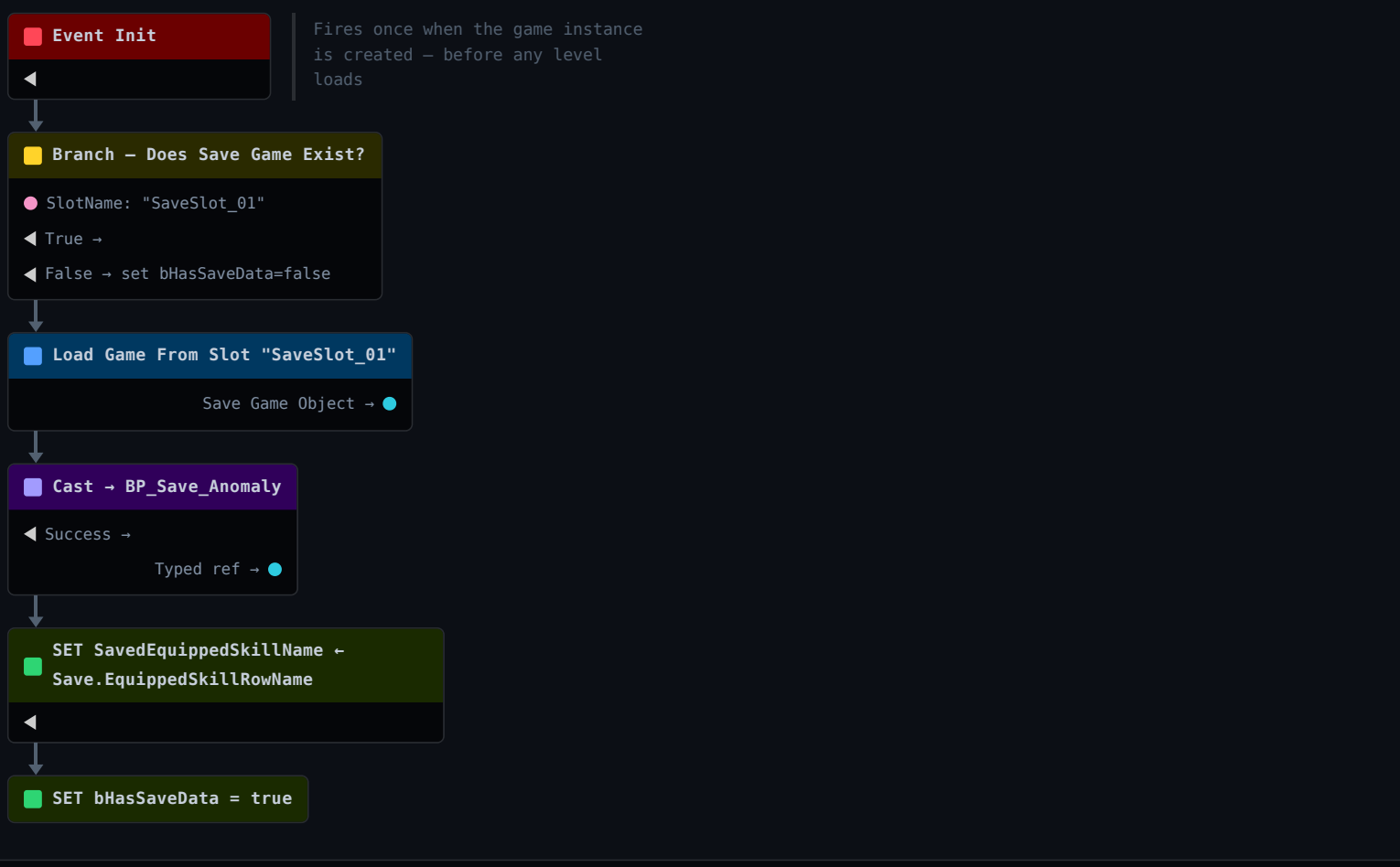
> The Game Instance persists across level loads and holds save data. The Game State holds read-only data accessible to all actors in the current level without direct references.

SETUP BP_GI_Anomaly — Game Instance

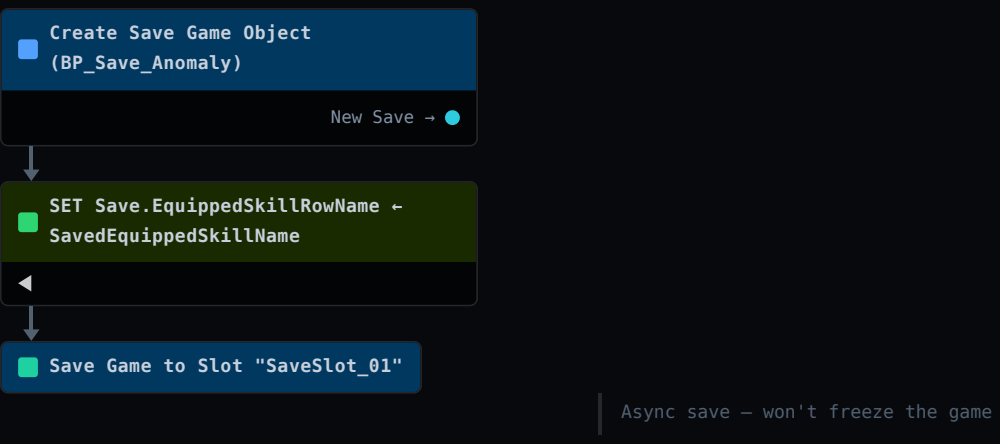
In `Core/GameInstances/` → Blueprint Class → parent = **GameInstance**. Name `BP_GI_Anomaly`. Open Project Settings → Maps & Modes → **Game Instance Class** → set to `BP_GI_Anomaly`.

Add variables: `SavedEquippedSkillName` (Name), `bHasSaveData` (Bool). These persist across level loads.

EVENT INIT — LOAD SAVE ON GAME START



SAVEGAME FUNCTION — CALLED FROM BP_SAVEPOINT AND AC_SKILLSLOTCOMPONENT



SETUP BP_GS_Anomaly — Game State

In `Core/GameModes/` → Blueprint Class → parent = **GameState**. Name `BP_GS_Anomaly`. Open `BP_GM_Anomaly` → in Class Defaults set **Game State Class** to `BP_GS_Anomaly`.

Add variables that any actor in the level needs to read globally without direct references:

VARIABLE	TYPE	WHO READS IT
CurrentWaveNumber	Int	WBP_HUD wave display, ArenaManager
bArenaActive	Bool	BP_SavePoint (CanInteract check — no save mid-combat)
TotalEnemiesRemaining	Int	HUD enemy counter display

💡 **Accessing the Game State from anywhere:** Right-click the graph → search **Get Game State** → Cast to `BP_GS_Anomaly`. Game State is accessible from any Actor, any Component, any Widget — no direct references needed.

FLOW SUMMARY — GAME INSTANCE + GAME STATE

1. **Game Instance vs Game State:** Game Instance persists across level loads (settings, save data, session state). Game State lives only for the current level but is accessible to every Actor in it without casting chains.
2. **Save flow:** BP_SavePoint calls BP_GI_Anomaly.SaveGame() → GI creates a fresh BP_Save_Anomaly, populates it from its own cached variables, and writes to disk. Load flow is the reverse: on Init, GI reads from disk and caches into its own variables. Player Character's BeginPlay reads from GI to restore equipped skill.
3. **bArenaActive in Game State:** BP_SavePoint checks this in CanInteract(). If an arena is active, CanInteract returns false — the player can't swap skills mid-combat. ArenaManager sets this to true on StartWave and false on OnArenaComplete.

🔧 DEBUG & TEST — GAME INSTANCE + GAME STATE

HOW TO TEST

- **Test GI persistence across levels:** Set a variable in BP_GI_Anomaly (e.g. SavedEquippedSkillName="TestSkill"). Open Level Streaming or use Open Level to switch maps. In the new level's BeginPlay, access the GI and print SavedEquippedSkillName. It should still be "TestSkill".
- **Test save/load round-trip:** Equip a skill, trigger a save. Close PIE. Look in `Saved/SaveGames/` in your project folder — a `SaveSlot_01.sav` file should exist. Start PIE again and print SavedEquippedSkillName from GI Event Init.
- **Test GS accessibility:** From a random enemy BP, call Get Game State → Cast to BP_GS_Anomaly → print bArenaActive. Should work without any direct reference chain.
- **Test CanInteract gate:** Start an arena (bArenaActive=true in GS). Approach save point — interact should be blocked. Confirm via print in CanInteract.

EXPECTED RESULTS

- ✓ PASS — GI variable persists after level change
- ✓ PASS — .sav file exists after save, GI loads it on next PIE
- ✓ PASS — GS accessible from any actor without cast chains
- ✓ PASS — Save point blocked during active arena
- ✗ FAIL — GI not found: check Project Settings → Maps & Modes → Game Instance Class is set to BP_GI_Anomaly. Without this, Get Game Instance returns the default base class.

BP_PLAYERCAMERA + BP_CAMERASHAKE_HIT

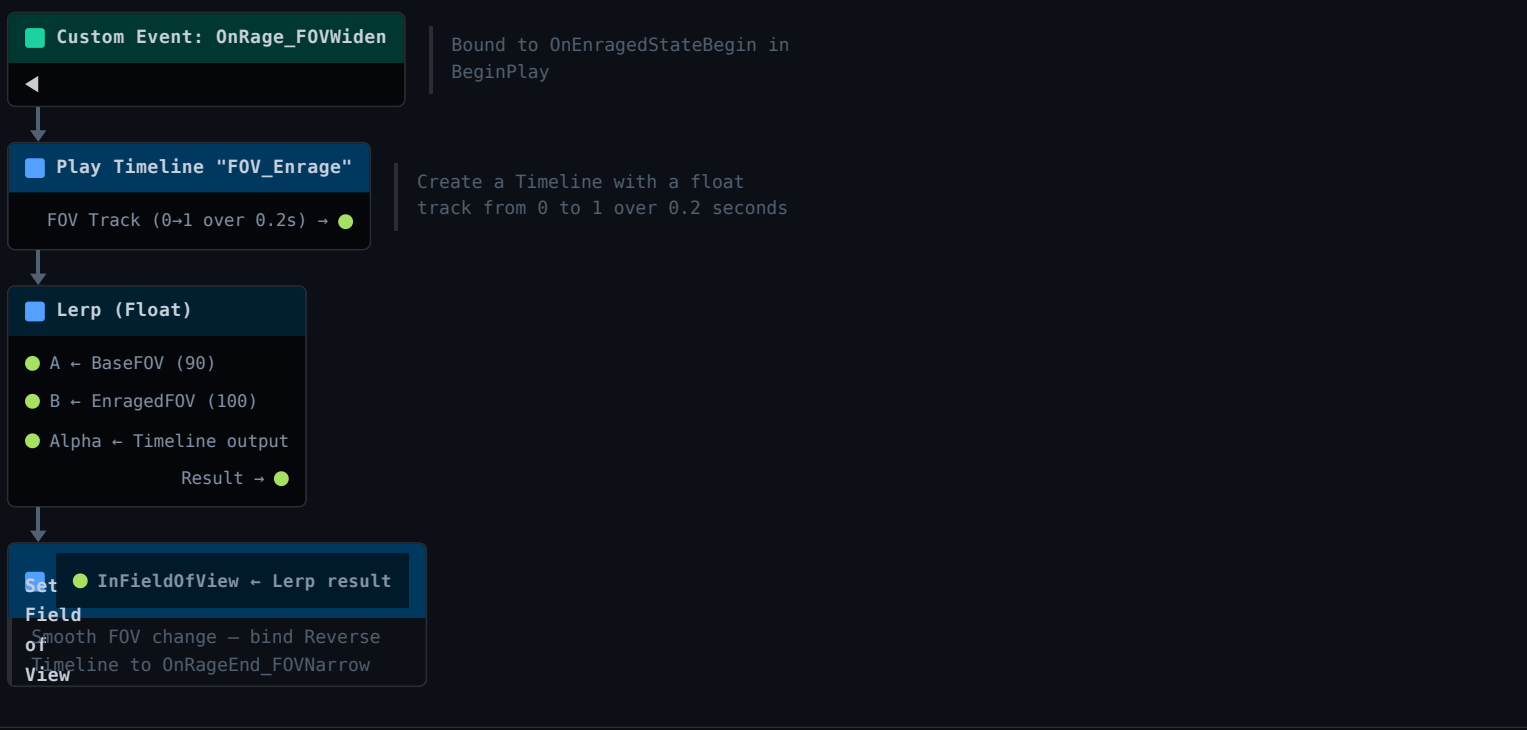
SETUP BP_PlayerCamera — spring arm and FOV management

In `Player/Camera/` → Blueprint Class → parent = Actor. In BP_PlayerCharacter's Components panel, add a **Spring Arm Component** and a **Camera Component** as a child of the spring arm. The camera setup happens directly in BP_PlayerCharacter — no separate BP_PlayerCamera Actor is strictly necessary unless you want to isolate camera logic. If you do use a separate BP, it would be a Camera Actor with variables:

VARIABLE	TYPE	PURPOSE
BaseFOV	Float	Default 90°
EnragedFOV	Float	Wider FOV during Enraged State, e.g. 100°
GrappleFOV	Float	Slightly wider during grapple, e.g. 95°

In BP_PlayerCharacter, bind to `OnEnragedStateBegin` and `OnEnragedStateEnd` to lerp the FOV. Use **FInterp To** on Tick (or a Timeline) to smoothly transition between BaseFOV and EnragedFOV:

FOV LERP DURING ENRAGED STATE — BIND THESE IN BEGINPLAY ALONGSIDE OTHER RAGE LISTENERS



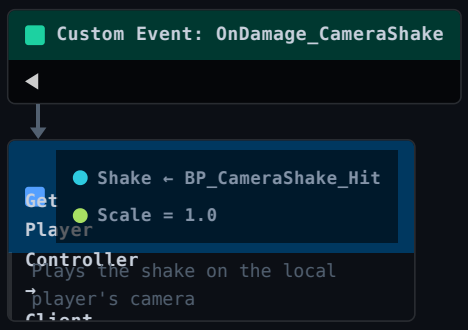
SETUP BP_CameraShake_Hit — damage feedback

In `Player/Camera/` → Blueprint Class → parent = **LegacyCameraShake**. Name `BP_CameraShake_Hit`. In Class Defaults, configure:

PROPERTY	VALUE	EFFECT
OscillationDuration	0.25s	Short, snappy shake
RotOscillation → Pitch → Amplitude	2.0	Vertical jolt
RotOscillation → Roll → Amplitude	1.5	Tilt on impact
LocOscillation → X → Amplitude	3.0	Forward/back push

To trigger the shake, go to AC_HealthComponent's `OnDamageTaken` event listener in BP_PlayerCharacter and add:

ONDAMAGETAKEN BOUND EVENT IN BP_PLAYERCHARACTER — PLAY CAMERA SHAKE



Bound to HealthComponent.OnDamageTaken in BeginPlay

FLOW SUMMARY — CAMERA SYSTEM

1. **FOV on Enrage:** Uses a Timeline for smooth interpolation instead of snapping. Timeline alpha (0→1) is used as the Lerp alpha between BaseFOV and EnragedFOV. Reversing the Timeline on state end gives a smooth return — same asset, two directions.
2. **Camera Shake setup:** LegacyCameraShake is the simplest UE shake — configure it in Class Defaults with no Blueprint graph needed. Values above give a short, impactful hit response without being nauseating.
3. **Decoupled triggering:** The shake is triggered by binding to HealthComponent.OnDamageTaken — same pattern as every other system. AC_HealthComponent never knows a camera shake exists.

🔧 DEBUG & TEST — CAMERA + CAMERA SHAKE

HOW TO TEST

- **Test FOV change on enrage:** Add a `Print String` every tick in the Timeline update event: print `"FOV: " + CameraComponent.FieldOfView`. Trigger enrage — the value should smoothly increase from 90 to 100 over 0.2s.
- **Test FOV revert on end:** After the enrage state ends, FOV should smoothly return to 90. If it snaps instead of smoothing, the Reverse Timeline call isn't connected to OnEnragedStateEnd.
- **Test camera shake on hit:** Have an enemy hit you. The screen should visibly jolt. If there's no shake, confirm Client Start Camera Shake is being called and that BP_CameraShake_Hit is assigned to the Shake pin.
- **Test shake intensity:** Temporarily increase all Amplitude values to 20. Take damage — the shake should be very obvious. Reduce back to normal values. Fine-tune for feel.

EXPECTED RESULTS

- ✓ PASS — FOV increases from 90 → 100 smoothly over 0.2s on enrage
- ✓ PASS — FOV returns 100 → 90 smoothly on state end
- ✓ PASS — Screen jolts visibly when taking damage
- ✗ FAIL — FOV doesn't change: the Timeline is being played but Set Field of View is not wired. Connect the Timeline float output → Lerp Alpha → Lerp result → Set FOV on the Camera Component.
- ✗ FAIL — No camera shake: confirm the player has a Camera Component attached. Client Start Camera Shake requires a valid camera to apply the shake to.

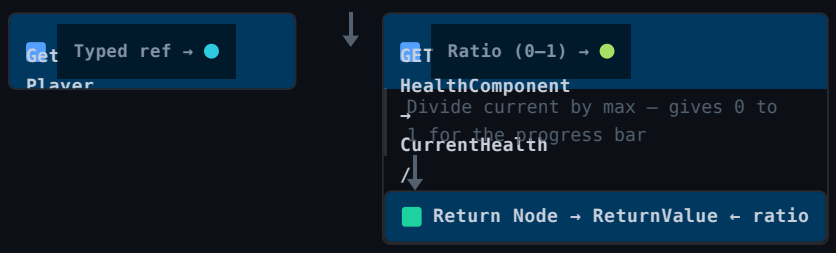
REMAINING HUD WIDGETS

WBP_HealthBar, WBP_AmmoDisplay, WBP_SkillSlotDisplay, WBP_WeaponWheel. All follow the same binding pattern from Part 1 — property bindings for continuously-updating values, event dispatcher bindings for discrete state changes.

WBP_HEALTHBAR Bind health percent to progress bar

Add a Progress Bar widget. Select it → Details → Percent → Bind → Create Binding. In the binding function:

HEALTHBAR PERCENT BINDING — TICKS EVERY FRAME

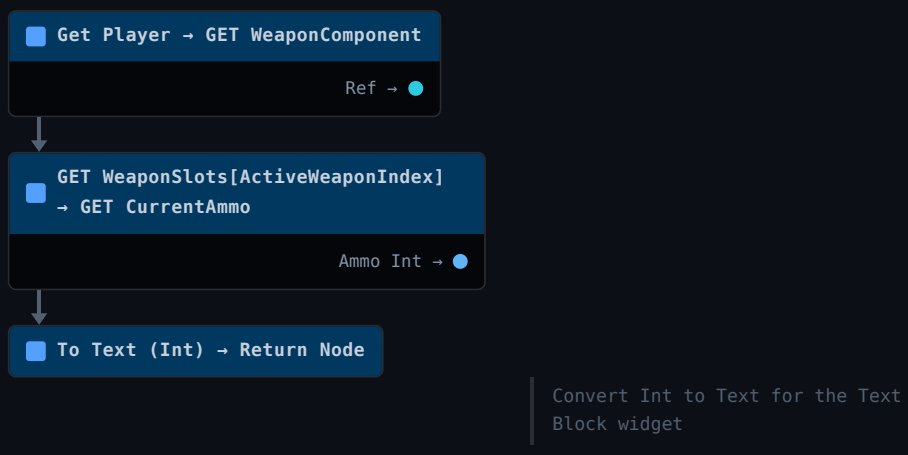


Also bind the **Fill Color** property: interpolate between green (full health) and red (low health) using a **Linear Color Lerp** node, with CurrentHealth/MaxHealth as the alpha. At 20% health the bar should be fully red.

WBP_AMMODISPLAY Show current ammo and active weapon

Add a Text Block for current ammo and one for max ammo. Bind each to a function. Also listen to the WeaponComponent's `OnWeaponSwitched` dispatcher to update the weapon icon:

CURRENTAMMO TEXT BINDING



WBP_SKILLSLOTDISPLAY

Show equipped skill icon + availability

Add an Image widget for the skill icon and a coloured overlay for availability state. In WBP_HUD's Event Construct (runs once when widget is created), bind to the player's SkillSlotComponent dispatchers:

EVENT CONSTRUCT — BIND TO SKILL EVENTS ONCE ON WIDGET CREATION

Get Player → GET SkillSlotComponent
Ref →

↓

Bind to OnSkillEquipped
Event ← Custom Event: RefreshSkillIcon

REFRESHSKILLICON CUSTOM EVENT

GET SkillSlotComponent
Texture →

↓

SET

Update the displayed icon to the newly equipped skill's texture

AVAILABILITY OVERLAY — BIND TO RAGE EVENTS

Bind to RageComponent.OnEnragedStateBegin
Event ← OnRageBegin_SkillGlow

↓

SET

During Enraged State, brighten the skill icon to show it's usable

WBP_WEAPONWHEEL Radial weapon selection overlay

This widget shows on hold of IA_WeaponWheel and closes on release. Add 3 Image/Button widgets arranged in a horizontal row or radially. In BP_PlayerController, bind IA_WeaponWheel Started/Completed:

IA_WEAPONWHEEL INPUT — SHOW/HIDE THE WHEEL AND SELECT WEAPON

IA_WeaponWheel — Started

Create Widget WBP_WeaponWheel → Add to Viewport (ZOrder=1)

Opens over the HUD

Set

Show cursor so player can hover/click weapon slots

IA_WEAPONWHEEL — COMPLETED (BUTTON RELEASED)

WeaponComponent.SwitchWeapon(highlighted index)

Index — last hovered slot (0/1/2)

Commit to whichever slot the cursor was on when released

Remove

Set

Return to gameplay input mode

FLOW SUMMARY — REMAINING HUD WIDGETS

1. **WBP_HealthBar:** Property binding returns CurrentHealth/MaxHealth as a float 0–1. Fill Color binding uses a Lerp between green and red — below 20% health the bar is fully red. Both bindings tick every frame but are very cheap float divisions.
2. **WBP_AmmoDisplay:** Property binding reads the current weapon's ammo from WeaponComponent. Updates automatically when the active weapon changes because ActiveWeaponIndex changes and the binding re-evaluates.
3. **WBP_SkillSlotDisplay:** Uses event bindings instead of property bindings — OnSkillEquipped fires rarely (only at save points), so polling every frame is wasteful. The icon only updates when the event fires. Rage state changes the glow overlay colour.
4. **WBP_WeaponWheel:** Created and destroyed on hold/release — not kept alive. The hover state of the three weapon buttons is tracked via OnMouseEnter events on each button, storing the last hovered index. On release, that index is passed to SwitchWeapon.

🔧 DEBUG & TEST — REMAINING HUD WIDGETS

HOW TO TEST

- **Test health bar colour shift:** Set MaxHealth=100. Take 85 damage (CurrentHealth=15, below 20%). The bar fill colour should be fully red. Restore health to 100 — should return to green.
- **Test ammo display on weapon switch:** Switch to Shotgun. Ammo display should show Shotgun's MaxAmmo. Fire a shot — count decreases. Switch to Pistol — display should immediately jump to Pistol's ammo count.
- **Test WeaponWheel open/close:** Hold the weapon wheel key. WBP_WeaponWheel should appear and mouse cursor should become visible. Release — menu closes, cursor hides, game input restores.
- **Use Widget Reflector:** Window → Developer Tools → Widget Reflector. Enable "Pick Hit-testable Widgets" and hover over the HUD to confirm each widget region is correctly registering input and has the right ZOrder.

EXPECTED RESULTS

- ✓ PASS — Health bar is fully red at 15 HP, green at 100 HP
- ✓ PASS — Ammo display updates immediately on weapon switch
- ✓ PASS — Weapon wheel: opens on hold, closes on release, cursor toggles correctly
- ✗ FAIL — Ammo always shows weapon 0's ammo: ActiveWeaponIndex isn't being read. The binding function must read WeaponSlots[ActiveWeaponIndex], not WeaponSlots[0].
- ✗ FAIL — Cursor stays visible after wheel closes: Set Input Mode Game Only is missing on the Completed binding.

DA_WEAPONDATA — PRIMARY DATA ASSET

DA_WeaponData is a Primary Data Asset — a per-weapon config object that holds everything a weapon needs: mesh, stats, sounds. Each weapon BP has one assigned in its defaults. This is the alternative to DataTable rows for assets that need to hold object references (meshes, sounds) rather than just numbers.

SETUP Create the DA_WeaponData class

First create a parent class: in `Weapons/Base/` → Blueprint Class → parent = **PrimaryDataAsset**. Name it `BP_WeaponDataAsset`. Add variables:

VARIABLE	TYPE	NOTES
WeaponName	Text	Display name
BaseDamage	Float	Per-shot base damage
MaxAmmo	Int	
ReloadTime	Float	Seconds
WeaponMesh	Soft Static Mesh ref	Async loaded — won't stall game
FireSFX	Soft Sound Base ref	
ReloadSFX	Soft Sound Base ref	
WeaponIcon	Soft Texture2D ref	For WBP_AmmoDisplay and WBP_WeaponWheel

Then create three instances: right-click in `Weapons/` → Miscellaneous → **Data Asset** → pick `BP_WeaponDataAsset`. Name them `DA_Weapon_Pistol`, `DA_Weapon_Shotgun`, `DA_Weapon_AssaultRifle`. Fill in the values for each.

In each weapon BP's Class Defaults, add a variable `WeaponData` (BP_WeaponDataAsset ref) and assign the matching DA asset. In BeginPlay, read the data asset to populate ammo and damage variables.

FLOW SUMMARY — DA_WEAPONDATA

1. **DataTable vs Data Asset:** DT_Weapons (from Part 1) is great for pure numerical data. DA_WeaponData is better when you need actual asset references (meshes, sounds, textures) alongside the numbers — DataTable cells can't easily hold soft asset references to specific content.
2. **Soft references:** Using TSoftObjectPtr (soft refs) for the mesh and audio means UE won't load the entire weapon mesh into memory until it's actually needed. Call Async Load Asset on them when the weapon is first equipped.
3. **Per-weapon instance:** Each of the three weapon BPs has its own DA asset assigned in defaults. BeginPlay reads from it once and stores values locally. The DA is a config file, not something queried at runtime repeatedly.

🔧 DEBUG & TEST — DA_WEAPONDATA

HOW TO TEST

- **Verify DA assignment:** Open BP_Weapon_Pistol → Class Defaults → find the WeaponData variable. It should show DA_Weapon_Pistol assigned. If null, the DA asset hasn't been set — drag it from the Content Browser onto the variable slot.
- **Test DA value loading:** In BP_WeaponBase BeginPlay, after reading the DA, print MaxAmmo. It should match the value in DA_Weapon_Pistol. Change it in the DA asset, restart PIE — the printed value should update without touching any Blueprint graph.
- **Test soft ref async loading:** If using soft object refs for mesh/audio, add a print before and after the Async Load Asset call. "Before" should print immediately; "After" may appear a few frames later as the asset streams in.
- **Test three different DAs:** Switch between all three weapons. Print WeaponData.WeaponName from each. Should print "Pistol", "Shotgun", "Assault Rifle" matching each DA's WeaponName field.

EXPECTED RESULTS

- ✓ PASS — DA_Weapon_Pistol assigned in BP_Weapon_Pistol Class Defaults
- ✓ PASS — DA value change reflects in game without BP recompile
- ✓ PASS — Each weapon prints its own DA name on switch
- ✗ FAIL — Weapon stats are defaults/zero: WeaponData variable is null. Ensure each weapon child BP has the correct DA assigned in its own Class Defaults (not inherited from the parent).
- ✗ FAIL — Mesh doesn't appear after async load: async callback isn't wired. Connect the Completed delegate of Async Load Asset to the Set Static Mesh node on the weapon's mesh component.